Volentify

System reference: what is built, how it works, and where every piece of data comes from

Component	State on 1 November 2026	Detail
Website	LIVE	https://volentify.vercel.app
API	LIVE	FastAPI on Vercel · /api/health reports ONLINE
Database	ONLINE	Neon PostgreSQL (read from DB_POSTGRES_URL)
Android app	LIVE	Volentify Mesh mesh-v1.0.5 · download at /mesh
Cyclone models	MEASURED	volentify-cyclone-ni@2026.11.01 · trained 2026-11-01
Dispatch benchmark	MEASURED	generated 2026-11-01 · reruns weekly
Research archive	LIVE	114 records so far · collected every 3 h
Google sign-in	LIVE	client configured; Google's sign-in screen verified, first real sign-in pending
Gemini	LIVE	gemini-3.5-flash · live briefing verified (about 1 s per call)
Session key	LIVE	NEXTAUTH_SECRET set in Vercel

Repository: github.com/Gyanvir/Volentify_pilot (public) · version 3.5.0 · commit 960434f. Figures in this document are read from the repository's model card and benchmark files and from the live site at the time of writing. Where something is a target rather than a measurement, it says so.

Contents

1. What Volentify is 3

2. Architecture 4

3. Technology used 5

4. Where the code lives 6

5. The website, page by page 7

6. The API, endpoint by endpoint 8

7. Where ML and AI are, and where they are not 9

8. The cyclone ML pipeline 10

9. Dispatch engine and its benchmark 11

10. Where every piece of data comes from 13

11. Storage and the research archive 14

12. Volentify Mesh: the Android relay app 15

13. Security, privacy, testing and operations 16

14. Where we are 18

1. What Volentify is

Volentify is a disaster-intelligence and volunteer-coordination platform for India. It gives incident commanders, NGOs, volunteers and citizens one shared picture of an emergency and keeps field reporting working when mobile networks fail.

It is built around four loops, and every feature belongs to one of them:

Loop	What happens	Built with
See	Map of active incidents, hazard zones, satellite imagery, radar, official alerts, nearby hospitals and shelters.	Tactical map (MapLibre), Satellite & bulletins page, NDMA SACHET feed, OpenStreetMap
Report	Citizens report from the web; volunteers report from phones, even with no network, through Bluetooth relaying.	Report form, Volentify Mesh Android app, mesh ingest API
Decide	Reports are triaged by language and urgency, duplicates merged, briefings written, cyclones forecast.	Laya triage, dedup, briefing, cyclone ML models, storm-surge estimator
Act	Missions are created and the right responders dispatched; specialists are protected from routine work.	Sperling dispatch engine, command hub, volunteer dispatch board, alerts

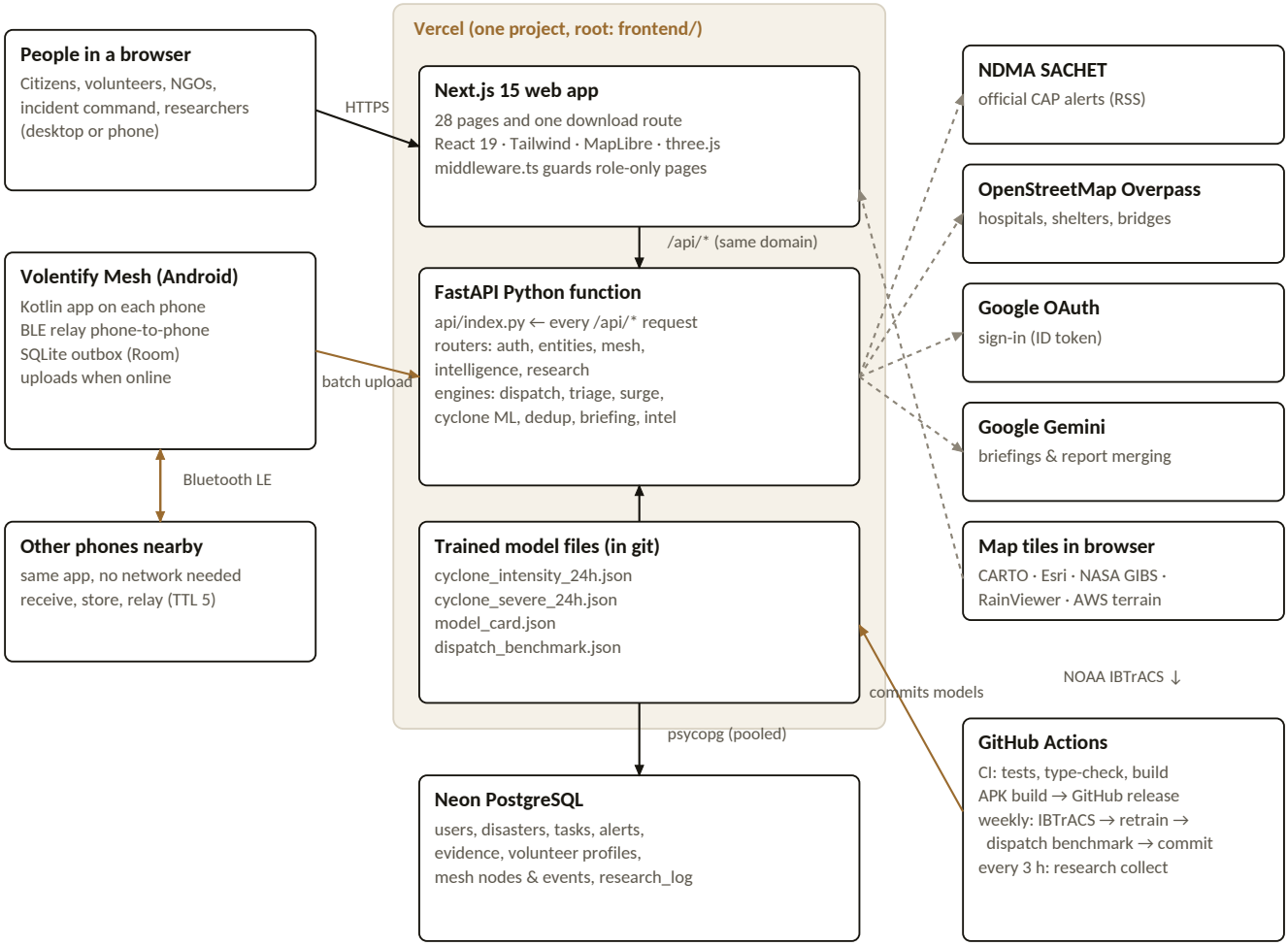
Who uses it

Role	What they can do	Verification
Citizen	See incidents, alerts and resources; submit reports; hold a civilian SOS pass.	None
Volunteer	Everything a citizen can, plus accept missions, set availability, get a gateway token for the mesh app, hold a responder muster pass.	None
NGO member	Command hub: review mesh reports, create missions, run dispatch, broadcast advisories.	Admin verifies
Incident admin	Everything, plus the admin console (users, roles, verification, system health) and research export.	Admin verifies (or listed in VOLENTIFY_ADMIN_EMAILS)
Researcher	Read-only analysis plus research archive export.	None

Every account completes a five-step onboarding (mobile number, role, role details, safety profile, review) and receives a signed digital muster pass.

2. Architecture

One Vercel project serves both the website and the API from the same domain, so there is no CORS, one deployment and one set of environment variables. The Android app is the only other runtime.



How a request flows

- A browser loads a Next.js page from Vercel. Pages fetch data from **/api/...** on the same domain.
- **vercel.json** rewrites every /api/* request to one Python function, **api/index.py** (FastAPI). It answers from PostgreSQL, calls the engines, or fetches an outside source.
- If the database is unreachable, reads fall back to a built-in demonstration dataset and every response carries an **X-Data-Source** header (database / fallback / memory) so screens can say "Demo dataset". Writes that must not be lost (mesh ingest) return 503 instead, so phones keep their outbox and retry.
- On the first connection to an empty database the API creates all tables and loads the starting dataset itself, under an advisory lock so parallel serverless instances cannot race.
- Phones relay reports to each other over Bluetooth; whichever phone reaches the internet uploads the batch to **/api/mesh/ingest** with a short-lived gateway token.
- GitHub Actions does all scheduled work: retraining, benchmarking, building the APK and collecting the research archive. Vercel redeploys automatically on every push to **main**.

3. Technology used

Layer	Technology	Version	Used for
Web framework	Next.js (App Router)	15.5	Pages, routing, server rendering, middleware
UI library	React	19.1	All interface components
Language	TypeScript	5.7	Type-checked front end
Styling	Tailwind CSS with CSS-variable tokens	3.4	Design system (docs/DESIGN.md)
Typography	Instrument Serif · Inter · IBM Plex Mono	Google Fonts	Headings · body · codes and numbers
Maps	MapLibre GL JS	4.7	2D/3D tactical map, layers, terrain, measuring, export
3D	three.js	0.171	God's Eye orbital globe on the home page and map HUD
Charts	Recharts	3.2	Analytics page
Icons · QR	lucide-react · qrcode	0.469 · 1.5	Icons; download QR code on /mesh
Route protection	jose (JWT verify at the edge)	5.9	middleware.ts role checks
API	FastAPI + Pydantic	≥0.115 · ≥2.10	All /api endpoints and request validation
API runtime	Python on Vercel serverless	3.12	api/index.py function
Database driver	psycopg 3 (binary)	≥3.2	PostgreSQL access, pooled-connection safe
Auth	PyJWT[crypto] · stdlib scrypt	≥2.9	Sessions, Google ID-token checks, passwords
HTTP client	httpx	≥0.27	SACHET, Overpass, Gemini, hosted triage
Database	PostgreSQL on Neon (via Vercel Storage)	15+	All durable data and the research archive
Schema	schema.sql (auto-applied) · Prisma schema	Prisma 6.19	Same schema two ways; Prisma optional
ML training	XGBoost · scikit-learn · pandas · NumPy	3.x · ≥1.5 · ≥2.2	Cyclone models, cross-validation
Optimisation	SciPy (Hungarian, HiGHS MILP)	≥1.13	Exact baselines in the dispatch benchmark
ML serving	treeinfer.py (pure Python)	—	Runs XGBoost trees without installing XGBoost
LLM	Google Gemini 3.5 Flash (auto-updating)	AI Studio or Vertex key	Briefing text and report merging
Android	Kotlin · Jetpack Compose · Room · WorkManager	2.0 · BOM 2024.09 · 2.6 · 2.9	Volentify Mesh app
Android build	Gradle · Android Gradle Plugin · KSP	8.9 · 8.5 · 2.0.20	APK build in GitHub Actions
Android networking	OkHttp · Android BLE GATT	4.12	Gateway upload · phone-to-phone relay
Hosting	Vercel	—	Web app + API, auto-deploy from GitHub
Automation	GitHub Actions	—	CI, APK releases, weekly training, archive collection
Tests	pytest · JUnit	—	API, ML pipeline and Android unit tests (section 13)

4. Where the code lives

```
Volentify_pilot/
├── README.md                    setup, deployment, research archive
├── docs/PRD.md                  product requirements v3.5 (the original specification)
├── docs/DESIGN.md               design system: tokens, type, map language, components
├── .github/workflows/
│   ├── ci.yml                   tests + type-check + production build on every push
│   ├── android.yml              builds, tests and releases the Mesh APK
│   ├── ml-train.yml             weekly: download IBTrACS, retrain, benchmark dispatch, commit
│   └── research-collect.yml     every 3 h: trigger /api/research/collect
├── ml/volentify_ml/
│   ├── data.py                  IBTrACS download and cleaning
│   ├── features.py              the 14 model features (order is a contract with the API)
│   ├── train.py                 training, storm-grouped evaluation, export
│   └── dispatch_bench.py        dispatch benchmark (Sperling vs exact methods)
├── mesh-android/                Volentify Mesh: Kotlin app (data, mesh, sync, ui, location)
└── frontend/                    the deployable app (Vercel root directory)
    ├── app/                     Next.js pages, grouped by purpose; map; mesh, mesh
    ├── components/              ui primitives, layout, map, globe, auth, operations
    ├── lib/                     API client, session, types, constants, release lookup
    ├── middleware.ts            role-based page protection
    ├── prisma/schema.prisma     database schema (Prisma form)
    ├── api/index.py             FastAPI entry point and /api/health
    ├── api/_lib/core/           config, database + fallback, security, research archive
    ├── api/_lib/engines/        dispatch, triage, surge, cyclone ML, dedup, briefing, intel …
    ├── api/_lib/routers/        auth, entities, mesh, intelligence, research
    ├── api/_lib/data/           schema.sql, seed, fallback dataset
    ├── api/_lib/models/         trained models, model card, dispatch benchmark (written by CI)
    └── tests/test_api.py        API acceptance tests
```

5. The website, page by page

Pages are grouped in the code by purpose. Access is enforced twice: by middleware.ts before the page loads, and again by the API for every action.

Route	Page	What it does	Access
/	Home	What Volentify is, live incident count, God's Eye globe, entry points, app download.	Anyone
/map	Tactical map	Full-screen 2D/3D map: incidents, hazard zones, missions, responders, mesh nodes; imagery base maps, rain radar, 3D terrain and hillshade, filters, distance measuring, PNG export.	Anyone
/disasters	Active incidents	Every tracked disaster, ordered by severity.	Anyone
/disasters/[id]	Incident dossier	Evidence timeline, missions, live situation briefing, forecast for the incident.	Anyone
/alerts	Alerts	Advisories issued by incident command.	Anyone
/volunteer	Volunteer dispatch	Open missions, accept a mission, set availability, run and explain a dispatch cycle.	Anyone (actions need sign-in)
/resources	Emergency resources	Helplines, hospitals with ICU capacity, shelters, and nearby infrastructure from OpenStreetMap.	Anyone
/predictions	Forecasts	Storm-surge sandbox, cyclone model performance, dispatch benchmark, incident forecasts, triage playground.	Anyone
/disaster-intelligence	Satellite & bulletins	Satellite imagery catalogue and preview, NDMA SACHET bulletins, report de-duplication tool.	Anyone
/analytics	Analytics	Charts computed live from incidents, missions, responders and mesh reports.	Anyone
/mesh	Mesh app download	Download button, QR code, install steps, troubleshooting.	Anyone
/download/android	(file)	Streams the latest APK from the GitHub release with the Android MIME type.	Anyone
/login · /register	Sign in · create account	Email + password or Google.	Anyone
/auth/callback	Google return	Completes Google sign-in (nonce + state checked).	Anyone
/onboarding	Muster pass	Five-step wizard; issues the signed digital pass.	Signed in
/dashboard	Dashboard	Role-specific desk: missions, alerts, muster pass, mesh gateway token.	Signed in
/agency	Command hub	Mesh review queue, mission form, dispatch run, advisory broadcast.	Verified NGO / admin
/admin	Administration	Users, roles, verification, database and system status.	Verified admin
/contact	Report an incident	Citizen report form with location; triaged on arrival.	Anyone
/research	Research	Methods, with each figure labelled measured, target or planned.	Anyone
/about · /faqs · /news · /knowledge	Information	About, questions, situation reports, preparedness guidance.	Anyone
/government · /ngo-partners	Partners	How agencies and NGOs work with Volentify.	Anyone
/privacy · /terms	Legal	Privacy notice and terms of use.	Anyone

6. The API, endpoint by endpoint

All endpoints live under /api on the same domain. Interactive documentation is generated at /api/docs. Requests are validated by Pydantic; errors come back as one readable sentence.

Method · path	Purpose	Auth
GET /api/health	Status, database state, which configuration variables are in use; ?deep=1 also tests Gemini live.	—
POST /api/auth/register · /login · /logout	Email accounts (scrypt passwords); sets the session cookie.	—
GET /api/auth/google/start · POST /api/auth/google	Google OpenID Connect sign-in with nonce and state.	—
GET /api/auth/me · POST /api/auth/onboard	Current user; complete onboarding and issue the muster pass.	Session
GET /api/admin/users · PATCH /api/admin/users/{id}	List users; change role or verification.	Admin
GET /api/disasters · /disasters/{id}	Incidents; one incident with evidence, tasks and forecast.	—
GET /api/disasters/{id}/briefing	Situation briefing for an incident.	—
GET · POST /api/tasks · POST /tasks/{id}/accept · PATCH /tasks/{id}	Missions: list, create, accept, update.	Varies
GET · POST /api/alerts	Advisories: list; issue (command roles).	Varies
GET /api/volunteers · /volunteers/me · POST /volunteer/availability	Responders and availability.	Varies
GET /api/organizations · /api/resources	Partner organisations; hospitals, shelters, helplines.	—
POST /api/reports	Citizen report: triaged and linked to the nearest active incident.	— (rate-limited)
POST /api/predict	Storm surge + hazard probability, driven by the trained cyclone models.	—
GET /api/ml/models	Model card, training data span, held-out metrics and the dispatch benchmark.	—
POST /api/triage/laya	Classify a free-text message: disaster type, severity, urgency, flags.	—
POST /api/events/ingest	De-duplicate a batch of raw reports into events.	—
POST /api/briefing/generate	Corroborated briefing from a set of evidence items.	—
POST /api/match · /api/match/dispatch-batch	Rank responders for one mission; run a full dispatch cycle.	—
POST /api/infrastructure/nearby	Hospitals, fire, police, shelters, assembly points, major bridges near a point (Overpass).	—
GET /api/bhuvan/layers · /api/bhuvan/bulletins	Satellite imagery catalogue; live NDMA SACHET alerts.	—
POST /api/mesh/token	Short-lived gateway token for a phone (240 min).	Volunteer / NGO / admin
POST /api/mesh/ingest	Phones upload relayed reports (≤100 per batch, ≤4 KB each, idempotent).	Gateway token
GET /api/mesh/events · /nodes · /incidents	Field reports (redacted for unverified readers), phones seen, incident areas to cache offline.	—
PATCH /api/mesh/events/{id}	Mark a report reviewed, or promote it to an official mission.	NGO / admin
POST /api/research/collect	Archive live CAP alerts, model version and a platform snapshot.	— (4 per hour)
GET /api/research/summary · /export	What the archive holds; download it as CSV or JSON Lines.	— · admin / researcher

7. Where ML and AI are, and where they are not

Only two components are trained machine-learning models. Everything else is an analytical model, an optimisation heuristic or a rule system, sometimes with an optional language model on top. This distinction matters for the paper and for how results are described.

Component	Kind	Learns from data?	Where it runs	State
Cyclone 24-hour intensity change	Trained ML (XGBoost regression)	Yes: IBTrACS, 1980 to now	API, every forecast	MEASURED
Severe-storm probability (≥48 kt in 24 h)	Trained ML (XGBoost classifier)	Yes: IBTrACS	API, every forecast	MEASURED
Storm-surge height	Physical estimator	No (calibrated formula)	API, /api/predict	TARGET
Hazard probability	Logistic combination, fixed weights	No; combines the ML probability with rain and surge terms	API, /api/predict	PARTIAL
Sperling dispatch	Optimisation heuristic	No (scoring rules)	API, every dispatch	MEASURED
Laya triage, tier 3	Multilingual lexicon + rules	No	API, every report	LIVE
Laya triage, tiers 1–2	Local router package / hosted model	External	Only if installed / IMPOSSIBL_API_KEY set	OPTIONAL
Report de-duplication	Spatio-temporal clustering	No (15 km / 2 h)	API	LIVE
Fact merging & briefing text	LLM (Gemini 3.5 Flash)	Pre-trained by Google, not by us	API, on briefing / merge	LIVE
Briefing fallback	Source weighting + conflict rules	No	API, if Gemini fails or takes over 12 s	LIVE
SitRep DistilBERT model	Planned trained ML	—	—	NOT BUILT

How the trained models reach users

- The weekly workflow trains with XGBoost and exports each model as a JSON tree dump.
- The API reads those files with **treeinfer.py**, a small pure-Python tree evaluator, so the Vercel function stays small (no XGBoost or NumPy at runtime). A parity test checks that it gives the same predictions as XGBoost; inputs and split thresholds are rounded to 32-bit floats exactly as XGBoost does.
- **/api/predict** converts the request to the 14 features, runs both models, then feeds the forecast peak wind into the surge estimator and the severe-storm probability into the hazard probability. The response names the model version, for example *volentify-cyclone-ni@2026.11.01 + surge-analytic-v1*.
- If the model files are missing, the API falls back to the analytical surge model alone and says so in the model name.

8. The cyclone ML pipeline

Step	What happens
1 · Download	NOAA NCEI IBTrACS v04r01, North Indian Ocean CSV, fetched fresh each week by ml/volentify_ml/data.py. The raw file is not kept (ml/data/ is git-ignored); the model card records its version and date span.
2 · Clean	Wind from RSMC New Delhi (IMD) where reported, otherwise WMO, otherwise US agencies; six-hourly fixes only; storms since 1980. This build: 463 storms, 8,558 fixes, 1980-10-10 to 2026-10-28.
3 · Features	14 features per fix (below). Their order is a contract shared with the API.
4 · Targets	intensity_24h: change in max sustained wind (kt) over the next 24 h. severe_24h: storm reaches ≥48 kt (IMD severe cyclonic storm) within 24 h.
5 · Augment	History masking: each fix is also trained with its 6/12/24-hour history hidden, so the model still works from a single report, as it must in the field.
6 · Evaluate	5-fold storm-grouped cross-validation (GroupKFold): every storm is held out whole, so scores describe cyclones the model has never seen. Compared with persistence and climatology.
7 · Export	Final models trained on all storms; written as JSON with model_card.json to frontend/api/_lib/models.
8 · Verify & ship	Pipeline tests and API tests run against the new files; the bot commits them; Vercel redeploys.
Schedule	Every Sunday night IST, and whenever the pipeline code or dispatch engine changes.

Features

Feature	Meaning	Feature	Meaning
wind_kt	current max sustained wind	dir_cos	heading (cosine)
pres_hpa	central pressure	month_sin	season (sine)
lat	latitude	month_cos	season (cosine)
lon	longitude	dwind_6h	wind change over last 6 h
dist2land_km	distance to land	dwind_12h	… 12 h
speed_kt	storm translation speed	dwind_24h	… 24 h
dir_sin	heading (sine)	arabian_sea	Arabian Sea (1) or Bay of Bengal (0)

Held-out results (this build)

Model	Metric	Model	Baseline	What it means
Intensity, 24 h	Mean absolute error	7.61 kt	9.67 kt (persistence)	21% lower error than assuming no change
Intensity, 24 h	MAE from one report only	8.03 kt	—	still works with no history
Severe within 24 h	ROC AUC	0.9592	0.5 (chance)	ranks storms that will turn severe very well
Severe within 24 h	AUC, storms still below 48 kt	0.8964	0.5	the hard, useful case: not yet severe
Severe within 24 h	Brier score	0.0579	0.1748 (climatology)	Brier skill 0.6687; base rate 23%
Severe within 24 h	Accuracy at 0.5	92.1%	77.4% (always "no")	accuracy alone flatters; prefer AUC and Brier

Samples: 6,818 intensity cases and 8,093 severe-storm cases. Trained with XGBoost 3.4.1.

9. Dispatch engine and its benchmark

The Sperling engine assigns open missions to available responders in strict priority order (critical, then high, moderate, low). Within a tier, each slot goes to the eligible responder with the best score:

```
C(i,j) = W_urgency · S_k · Sim · Π_squander × 1/(1 + d/d0) × 1/(1 + β·H_work)
```

Term	Value in code	Meaning
W_urgency	10 / 5 / 2 / 1	critical / high / moderate / low
S_k	1/√N_k	scarcity: the fewer responders hold skill k, the higher the weight
Sim	1.00 / 0.55 / 0.10	exact skill / adjacent skill (e.g. paramedic ↔ first aid) / generic
Π_squander	0.12	a rare specialist (skill held by ≤3 responders) sent to generic work scores ×0.12
d0	15 km	distance at which the score halves
β	0.15	fatigue: each active mission lowers the score
Eligibility	rule	specialist missions never go to untrained responders

Benchmark method (reruns every week)

ml/volentify_ml/dispatch_bench.py calls the production engine unchanged and compares it with three other methods on synthetic musters around one Indian incident area: generic skills are common, specialist skills rare, and there are about 1.1 mission slots per responder. Five scenarios per size; median time on the GitHub Actions runner. The exact MILP has a 10-second limit.

- **Nearest responder**: missions in arrival order, nearest eligible free person: how a busy human coordinator works.
- **Exact optimum**: the Hungarian algorithm maximises the total match score, but ignores urgency order.
- **Exact priority MILP**: HiGHS solves the true lexicographic optimum (all critical first, then high, …).

Results generated 2026-11-01 by the weekly workflow (dispatch_benchmark.json); the website shows the same table on /predictions.

Scale	Method	Time / cycle	Critical filled	Urgency-weighted fill	Score vs optimum	Specialists on generic	Timed out
40 responders · 46 slots	**Volentify (Sperling)**	1.6 ms	88%	83%	92%	14%	0%
	Nearest responder	0.4 ms	52%	69%	58%	34%	0%
	Exact optimum	3.0 ms	86%	83%	100%	14%	0%
	Exact priority MILP	15 ms	88%	83%	99%	14%	0%
150 responders · 169 slots	**Volentify (Sperling)**	17 ms	100%	90%	92%	0%	0%
	Nearest responder	5.2 ms	64%	75%	56%	13%	0%
	Exact optimum	39 ms	100%	91%	100%	0%	0%
	Exact priority MILP	249 ms	100%	91%	99%	0%	0%
500 responders · 544 slots	**Volentify (Sperling)**	179 ms	100%	94%	91%	0%	0%
	Nearest responder	55 ms	69%	77%	56%	0%	0%
	Exact optimum	412 ms	100%	94%	100%	0%	0%
	Exact priority MILP	5.7 s	100%	94%	100%	0%	0%
1000 responders · 1125 slots	**Volentify (Sperling)**	730 ms	100%	91%	90%	0%	0%
	Nearest responder	220 ms	67%	75%	55%	0%	0%
	Exact optimum	1.7 s	100%	91%	100%	0%	0%
	Exact priority MILP	15.3 s	—	—	—	—	100%

What the numbers say

- Sperling fills critical missions as well as the exact priority MILP wherever the MILP finishes (at 500 responders: 100% vs 100%), and far better than the nearest-responder baseline (67% at the largest scale, against 100%).
- Exact priority optimisation does not scale: at 1000 responders the MILP took 15.3 s and timed out in 100% of runs; at 500 it took 5.7 s. Sperling took 730 ms at the largest scale.
- Sperling gives up some total match score compared with the Hungarian optimum (column "Score vs optimum") because it deliberately serves urgent missions first, even when a less urgent assignment would score higher.
- **Supersedes the PRD figures.** The PRD quotes 0.32 ms, a 64.2% MILP timeout rate and 100% efficiency. No benchmark in the code produced those numbers. Use this table instead; it is regenerated every week.
- The benchmark found and fixed one bug: rare specialists who also held a generic skill were not protected from generic work. The engine now protects them, as the PRD specifies.

10. Where every piece of data comes from

"Live" means fetched at the moment someone asks. "Archived" means also written to the research archive.

Source	What we take	How and when	Stored?
NOAA NCEI IBTrACS v04r01	North Indian Ocean cyclone best tracks since 1980	Downloaded weekly by the training workflow	Only the trained models and model card (in git)
NDMA SACHET	Official CAP alerts for India (RSS): title, text, severity, issuer, time	Live on /api/bhuvan/bulletins; every 3 h by the collector	Yes: research_log (cap_alert), each alert once
OpenStreetMap (Overpass API)	Hospitals, fire and police stations, shelters, assembly points, major bridges	Live, per request, around a point	No
Esri World Imagery	High-resolution satellite base map	Tiles loaded by the browser	No
NASA GIBS	MODIS Terra and VIIRS SNPP true-colour daily imagery	Tiles loaded by the browser (yesterday's date)	No
RainViewer	Precipitation radar composite	Browser fetches the latest frame index	No
CARTO	Dark Matter vector base map	Browser	No
AWS Terrain Tiles (Terrarium)	Elevation for 3D terrain and hillshade	Browser	No
Google	Sign-in identity (name, email, picture)	On sign-in; ID token verified against Google's keys	users table
Google Gemini	Generated briefing text, merged facts	When a briefing or merge is requested (key needed)	No
Impossibl hosted triage (optional)	Triage classification	Only if IMPOSSIBL_API_KEY is set	No
Citizens (web)	Incident reports with location	Report form → /api/reports	evidence table
Volunteers' phones	Mesh reports and messages, phone battery and position	Bluetooth relay → gateway upload	mesh_events, mesh_nodes
Responders and organisations	Profiles, skills, availability, missions	Onboarding, dashboards, command hub	users, volunteer_profiles, tasks, organizations
Starting dataset	5 incidents (Brahmaputra flood, Cyclone Remal, Wayanad slope risk, Garhwal forest fires, heatwave), 15 evidence items, 7 missions, 9 responders, 5 alerts, 5 hospitals, 4 shelters, 8 helplines, 5 organisations	Loaded once into an empty database; also the offline fallback	Database (seed) and code

The seeded incidents are illustrative, built around real places (for example the Brahmaputra at Guwahati and a cyclone track near Puri). They are not live disasters, and screens label them as demonstration data when served from the fallback.

ll. Storage and the research archive

Where	What is kept	How long
Neon PostgreSQL	organizations, users, volunteer_profiles, disasters, tasks, evidence, alerts, mesh_nodes, mesh_events, research_log	Until deleted
Git (GitHub)	Code, trained models, model card, dispatch benchmark; every weekly retrain is a commit, so model history is preserved	Permanent
GitHub releases	Every Mesh APK build (mesh-v1.0.N)	Permanent
Each phone (SQLite via Room)	Reports and messages the phone created or relayed; settings; cached incident areas	Until uninstalled
Browser	Session cookie (HTTP-only, 72 h); OAuth nonce/state cookies (10 min)	Short-lived
Fallback dataset (code)	Demonstration data served when no database is reachable	Static

Research archive (research_log)

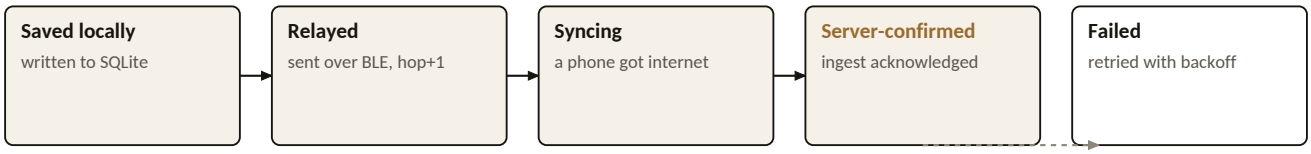
Live observations and model outputs are archived with timestamps so results can be reproduced and analysed later. Writes are best-effort and never slow down the request that triggered them. No personal data is archived: no report text, names, phone numbers or exact volunteer locations.

Kind	What is recorded	When	Records so far
cap_alert	Each NDMA SACHET alert, once (fingerprint of title, time, link)	every feed pull and every 3 h	102
forecast	Inputs and full output of every /api/predict call, with model version	every forecast	7
dispatch_run	Tier counts, completeness, latency, scarcity weights, score distribution	every dispatch cycle	1
model_version	Model card of each trained version	once per version	2
platform_snapshot	Active incidents, task and responder status counts, mesh nodes, hop counts	every 3 h	2

Read it at /api/research/summary. Export with /api/research/export?kind=…&format=csv (or jsonl), signed in as an incident admin or researcher.

12. Volentify Mesh: the Android relay app

The core offline feature. Phones pass reports and messages to each other over Bluetooth Low Energy, with no mobile network or Wi-Fi. Any phone that reaches the internet uploads everything it holds.



Every report shows one of these states in the app. Phones keep the outbox until the server confirms.

Aspect	How it works
Discovery	Every phone advertises and scans for the Volentify BLE service (UUID 7f3c0001-6a1e-4c2b-9b8e-5d1f0c7a9e10) and acts as both GATT server and client.
Transfer	Envelopes are written to the inbox characteristic (7f3c0002-…) in frames with a 6-byte header, sized to fit any MTU, then reassembled.
Envelope	event id (UUID), origin node, type, priority, payload ≤4 KB, location, observed time, hop count, TTL. Validated on receipt.
Relaying	Duplicates dropped by event id. Each relay stores hop+1 and TTL−1 and forwards while TTL > 0 (maximum TTL 5). A report is never echoed back to the phone it came from.
Sessions	Up to 30 events per session, 25 s session timeout, 45 s cool-down per peer.
Report types	Person rescued, missing or trapped; medical emergency; fire; flood; resource request; volunteer status; disaster status; plus free-text Messages (≤500 characters, optional urgent flag).
Storage	SQLite (Room): survives closing the app and restarting the phone. Relay restarts after boot.
Upload	WorkManager syncs batches of up to 100 to /api/mesh/ingest with a gateway token from the web dashboard, retrying with exponential backoff until acknowledged.
Privacy	Node ids are random UUIDs, never hardware addresses. Location can be masked by ±500 m. Sensitive reports are redacted on the website for unverified viewers.
Screens	Home (status, Sync now), Report, Messages (chat-style), Settings (name, server, token, mask location).
Permissions	Nearby devices (Bluetooth scan, advertise, connect), location, notifications, foreground service (connected device), internet, start at boot.
Build	Kotlin 2.0, Compose, minSdk 26 (Android 8.0), targetSdk 34. GitHub Actions builds, tests and publishes each release; latest mesh-v1.0.5.
Install	From /mesh on the website (served by our own domain with the Android MIME type), or the QR code on desktop. Play Protect may ask to confirm: "More details → Install anyway".

13. Security, privacy, testing and operations

Security and privacy

- Sessions are HS256 JWTs in an HTTP-only, SameSite=Lax cookie (Secure in production), valid 72 hours. The signing key is NEXTAUTH_SECRET, or, if that is absent, a key derived from the database URL; the API and the page middleware derive the same key.
- Passwords are hashed with scrypt ($n=2^{14}$, r=8, p=1, 16-byte salt). Google sign-in verifies the ID token's signature, audience, issuer and a one-time nonce.
- Pages for elevated roles are protected in middleware.ts and every action is checked again in the API. NGO and admin roles need verification by an administrator.
- Rate limits per server instance: login 10/min, citizen reports 6/min, mesh ingest 60/min, research collection 4/hour.
- Field reports about people (missing, trapped, rescued, medical) are redacted for anyone who is not a verified NGO member or admin.
- Secrets live only in Vercel environment variables and the local .env (git-ignored). The Android signing keystore is on the developer's machine, not in git.

Tests

Suite	Covers	Runs
frontend/tests (38 tests)	Every API contract in PRD §14, auth, roles, mesh ingest durability, redaction, engines, configuration fallbacks, research archive	CI on every push; weekly after retraining
ml/tests (6 tests)	Data cleaning, features, training, tree-evaluator parity with XGBoost, dispatch benchmark sanity	CI on every push; weekly
mesh-android unit tests (7)	Envelope validation, framing and reassembly	APK workflow
Type-check + production build	Whole web app	CI on every push

Environment variables (Vercel)

Variable	Needed for	Now
DATABASE_URL / DB_URL / DB_POSTGRES_URL / POSTGRES_URL	Database (any one; Neon sets these)	LIVE
NEXTAUTH_SECRET	Session signing key (otherwise derived from the database URL)	LIVE
NEXTAUTH_URL	Public address for OAuth (localhost values are ignored in production)	LIVE
VOLENTIFY_ADMIN_EMAILS	Accounts that become verified admins on sign-up	OPTIONAL
GOOGLE_CLIENT_ID	Google sign-in	LIVE
GEMINI_API_KEY (+ GEMINI_MODEL)	Gemini (AI Studio or Vertex express key). Model defaults to gemini-3.5-flash; a retired model is replaced automatically and a slow one stepped down	LIVE
IMPOSSIBL_API_KEY	Hosted triage tier	OPTIONAL
MESH_RELEASE_REPO	Which GitHub repo the APK download reads (default Gyanvir/Volentify_pilot)	OPTIONAL

Automated workflows (GitHub Actions)

Workflow	Trigger	Does
CI	every push	API + ML tests, type-check, production build
Volentify Mesh APK	changes under mesh-android/	build, unit-test, sign, publish GitHub release
Train cyclone models	weekly (Sunday night IST), pipeline or engine changes	download IBTrACS, retrain, evaluate, benchmark dispatch, test, commit
Collect research archive	every 3 hours	POST /api/research/collect on the live site

14. Where we are

Done and live

Area	State
Website (28 pages), design system, tactical map with imagery, radar and 3D terrain	DONE
API (44 endpoints) with database, fallback dataset and durability rules	DONE
Neon database: auto-created schema, seed data, research archive	DONE
Email accounts, roles, onboarding, muster passes, admin verification	DONE
Cyclone ML models trained on real data, evaluated on held-out storms, retrained weekly	DONE
Dispatch engine with weekly benchmark against exact optimisation	DONE
Laya triage (lexicon tier), de-duplication, rule-based briefings	DONE
Live NDMA alerts, OpenStreetMap infrastructure, satellite and radar layers	DONE
Mesh Android app: BLE relay, messages, offline storage, gateway sync, releases, website download	DONE

Partial or waiting on configuration

Item	What is needed	State
Google sign-in	Configured and accepted by Google; complete one real sign-in to confirm the full round trip	ACTION
Stable APK signing	Add the four signing secrets from mesh-android/signing/HOW-TO-USE.txt; rebuild; uninstall the old build once	ACTION
Mesh field test	Two or three phones with mobile data off; confirm relay and later upload	ACTION
Storm-surge accuracy	No open surge observations for India; needs tide-gauge data (e.g. INCOIS) to validate	PARTIAL
Database password	The old Supabase password was shared in chat; that project is gone, but rotate anything reusing it	ACTION

Not built yet

Item	Note
SitRep DistilBERT language model	In the PRD as planned; triage currently uses the lexicon tier
Trained surge model	Blocked on observation data
iOS app	Only Android exists
Push / SMS alerts	Alerts are shown on the website; no SMS or push channel yet

For the research paper

- Cite the data: IBTrACS version and the date span in the model card; NDMA SACHET for alerts.
- Use only measured figures: the cyclone metrics (section 8) and the dispatch benchmark (section 9). Treat PRD performance numbers as targets.
- The research archive has been collecting since 1 November 2026; the longer it runs, the stronger any analysis of alerts, forecasts and dispatch behaviour will be.
- Check every reference before citing it. "Sperling (2026)" is used as a name in the code and PRD but has not been verified as a published source.